

# AIA: A 16nm Multicore SoC for Approximate Inference Acceleration Exploiting Non-normalized Knuth-Yao Sampling and Inter-Core Register Sharing

Shirui Zhao\*, Nimish Shah\*, Wannes Meert†, and Marian Verhelst\*

\*MICAS-ESAT, KU Leuven, †DTAI, KU Leuven  
{shirui.zhao, marian.verhelst}@kuleuven.be

**Abstract**—Probabilistic graphical models (PMs) are popular to empower machine learning with the ability of reasoning and decision-making. To perform approximate inference in PMs, sampling-based Markov Chain Monte Carlo (MCMC) algorithms are commonly employed. Unfortunately, MCMC is compute-intensive and hard to run in parallel, resulting in inefficient execution on modern CPU/GPU platforms. This paper proposes AIA, an Approximate Inference Accelerator designed to empower decision-making and reasoning at the edge. AIA consists of a RISC-V host, and a 2D mesh of 16 customized RISC-V cores optimized to efficiently support PM inference, each featuring (i) a novel non-normalized Knuth-Yao sampler and interpolation unit; and (ii) core-to-core direct data access via the register file, which provides solutions for compute-intensive operations. To fully exploit the parallel potential of Markov Chain Monte Carlo (MCMC) algorithms, a customized compiler chain has been developed for effective spatial mapping and scheduling on the chip. AIA can generate 1277 MSample/s at 0.9V and 20 GSamples/s/W at 0.7V which is up to  $2\times$  faster and  $1.45\times$  more energy efficient compared to the previous state-of-the-art Markov Random Field (MRF) accelerator. We further map Bayesian Networks benchmark onto AIA to show the flexibility of our design.

**Index Terms**—Probabilistic graphical models, Bayesian inference, Approximate inference, MCMC, Knuth-Yao sampling, RISC-V

## I. INTRODUCTION

Edge intelligence requires both perception and reasoning capabilities, also denoted as “hybrid AI”. As shown in Fig. 1, Deep neural networks (DNN) have brought enormous successes for perception tasks, yet proven to be ill-suited for reasoning tasks [1]. A more potent framework for addressing a spectrum of complex decision-making tasks is probabilistic graphical models (PM) [2], which allows for integrating human knowledge into the models and handling reasoning with uncertainties. For example, in a 3D scene application (Fig. 1)(a), the human prior knowledge about the size and relationship between objects in the 3D image facilitates a more accurate scene understanding [3]. In other fields such as finance [4], pharmaceuticals [5], etc., which require explainable “white-box” models, PMs are also preferred over their DNN counterparts for certain applications.

The PMs can be represented with (un)directed graphs, with nodes representing random variables (RVs) and edges indicating relationships between RVs. However, when scaling up the complexity of the graphs, the computing quickly becomes intractable. Instead of exact inference [6], MCMC techniques

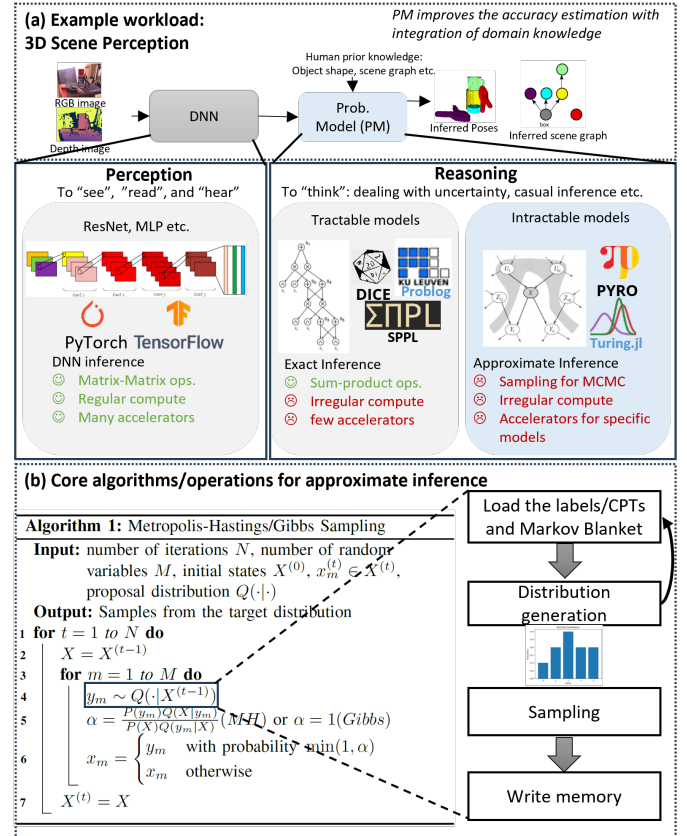


Fig. 1: Deep learning v.s. probabilistic machine learning.

offer a scalable solution by generating samples from the posterior distribution [7]. The core operation of MCMC is illustrated in Fig. 1(b). The parallelism is hard to utilize as it requires processes  $N$  iterations to update all RVs in the graph until convergence to the desired target distributions. Line 4 denotes the update of one RV, which needs ① irregular data access for its neighbors’ RVs in the Markov Blanket (MB), ② perform complex computing (exp, log, etc) to generate distribution for this RV, and ③ sample from the distribution. This results in very poor computing efficiency on CPU and GPU. Moreover, State-of-the-art neural network accelerators also do not match with the irregular graph-based traversal and computational kernels. As a result, specific ASIC accelerators

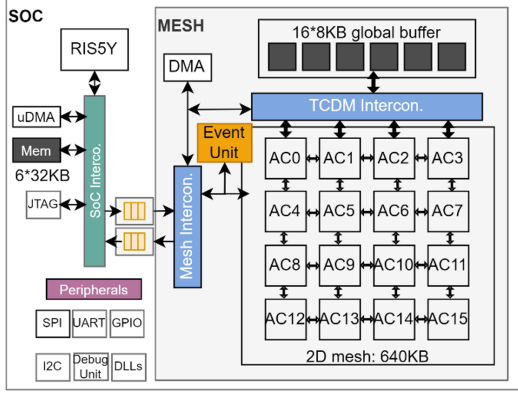


Fig. 2: The AIA architecture with 16 accelerator cores(AC)

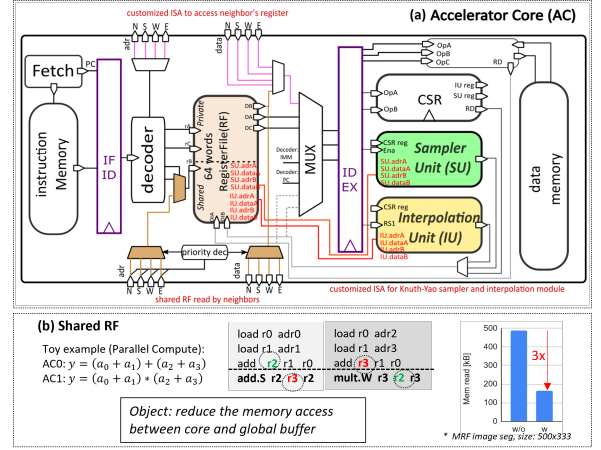


Fig. 3: (a) The AC micro-architecture; (b) Example of using inter-core register sharing ISA.

have been developed targeting the inference over PMs. The ASIC PM accelerators available in the SotA are highly customized, lacking software programmability to support multiple types of PMs [8] [9] or only support exact inference [6] which is not suitable for large PMs.

This paper presents AIA, the first silicon implementation of a multi-core programmable approximate inference accelerator with RISC-V ISA extensions. AIA boosts the efficiency of MCMC for various types of probabilistic graphical models, including Markov Random Field (MRF) and Bayesian Networks (BN).

## II. AIA HARDWARE DESIGN

Fig. 2 shows the 16nm AIA SoC with a RIS5Y host core [10] and a 4x4 mesh of customized accelerator cores (AC). The host core is responsible for connecting the accelerator to the outside over GPIO, SPI, I2C, Hyperam, and UART interface, which contains 192KB SoC memory for data buffering. The accelerator mesh sits in a separate clock domain and contains 4x4 ACs, an event unit for AC synchronization, and a tightly-coupled 128KB global buffer accessible by the top 4 ACs.

As shown in Fig. 3, each AC is a customized RISC-V core [10] with an 8KB instruction and a 32 KB data scratchpad, extended with A) neighboring core register access; and B) custom instructions for sampling and interpolation.

### A. Inter-core register sharing

MCMC frequently requires access to its neighbor RVs, hence benefiting from a 2D mesh architecture for data communication between processing cores dealing with different nodes in parallel. Each AC contains a register file (RF) with a size of 64 words. To enable this sharing without costly memory accesses, the ACs have been extended with single-cycle read access capabilities into 32 shared registers of their N-E-S-W neighbors, as shown in Fig. 3(a). A simple example in Fig. 3(b) shows the benefit of using the RF sharing feature. It can avoid the global buffer operation instruction after computing. Given the MRF image segmentation task, we can reduce 3x memory read overhead via this approach.

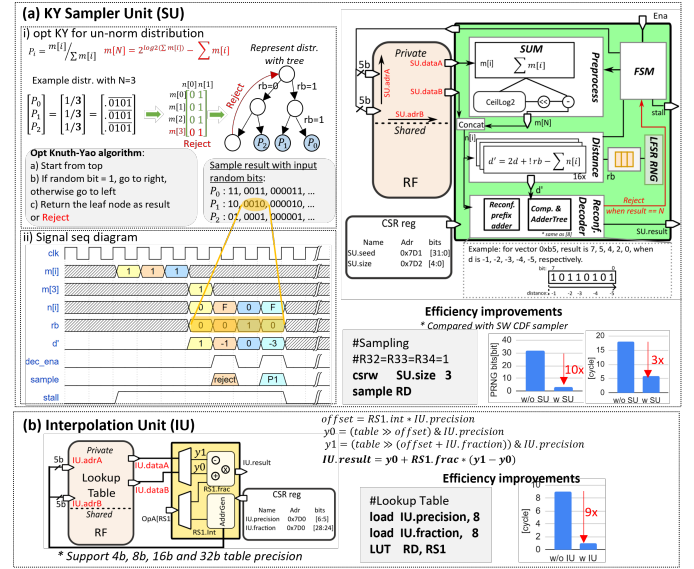


Fig. 4: (a) SU for non-normalized integer distributions; (b) IU for LUT function of complex operations.

### B. ISA extension for sampling and interpolation

Sampling from arbitrary discrete distributions is the key operation in the MCMC algorithm. In existing works [2] [8] [9], the traditional cumulative distribution function(CDF) samplers are widely used, which shows low performance compared to the Knuth-Yao (KY) algorithm [11] [12]. We propose a novel KY hardware sampler unit(SU) with rejection feature. The algorithm is described in Fig. 4(a). It starts from the root node and traverses the tree using random bits from Linear Feedback Shift Registers (LFSR). When it reaches a leaf node, it returns the corresponding sample result. Unlike the initial KY sampler, the novel one will restart when reaching the rejected item. This method effectively reduces the need for extensive cumulative and normalization operations while simultaneously minimizing the memory footprint required for the distribution.

The SU is integrated into the AC's decode and execution

TABLE I: Customized RISC-V ISA

| Function   | Func8<br>31:29 | Func7<br>28:25 | Func3<br>14:12         |
|------------|----------------|----------------|------------------------|
| private RF | 0              | 0-9            | RD[5], RS1[5], RS2[5]  |
| shared RF  | 1              | 0-9            | 0:left shared RF, etc. |
| SU         | 2              | 0x0            | 0                      |
| IU         | 3              | 0x0            | 0                      |

stage. After distribution generation, the non-normalized distribution is stored in RF in a row-wise way. The KY sampling algorithm requires column-wise access to the probabilities for tree search after computing the rejected item. To this end, we add 2 additional read ports to the RF (a row-wise and a column-wise one). The sampling instruction requires a variable number of clock cycles, during which the rest of the processor pipeline is automatically stalled. An example of sampling from the distribution with  $P_x = \frac{1}{3}$ , for  $x \in [0, 2]$  is shown in Fig. 4(a). It consumes an average of 3 random bits per sample, resulting in a 3x reduction in runtime compared to the CDF sampler.

A second ISA extension is introduced for look-up-table (LUT)-based interpolation unit (IU). MCMC requires repetitively for complex operations (exp, log, etc.) in the distribution generation stage of all RVs. This single-cycle datapath extension also requires two additional RF read ports, directly accessible in the EX stage based on the interpolation index fetched from the register file through RS1. As shown in Fig. 4(b), the *offset* and *IU.address* are generated according to the integer part of the input data. Following the formula in the figure, the linear interpolation result is calculated in one cycle. Compared to memory-based LUT, it can save 9x runtime.

The customized ISA are defined in Table I. As the SU/IU requires an enlarged RF address space for its private RF and neighbors' shared RF, the address index width is extended by 3 bits for the customized arithmetic R-type operations. Also, we added two instructions to trigger the IU and SU.

### III. CUSTOMIZED COMPILER

A custom compiler chain (Fig. 5) is developed for AIA, to enable rapid mapping of MCMC algorithms for a wide variety of PMs.

The PMs can be described using a probabilistic programming language (PPL): aGrums [2], then subsequently go through a fixed-point model quantization stage [7] with negligible accuracy loss (see Fig. 5), and a graph coloring stage. Graph coloring splits all nodes (variables) of the models into sets of conditionally independent variables (colors), which can be updated in parallel. E.g. for MRF, block Gibbs sampling requires only two colors in a checkerboard pattern [8] [9]. Irregular workloads, such as Bayesian Networks [4] [5], typically require more sets. We utilize aGrum [2] to convert the Directed Acyclic Graph (DAG) to the factor graph and exploit the heuristic DSatur [13] to color the graph.

Afterward, mutually independent nodes are mapped onto the 16 parallel AC cores, using the information from the graph

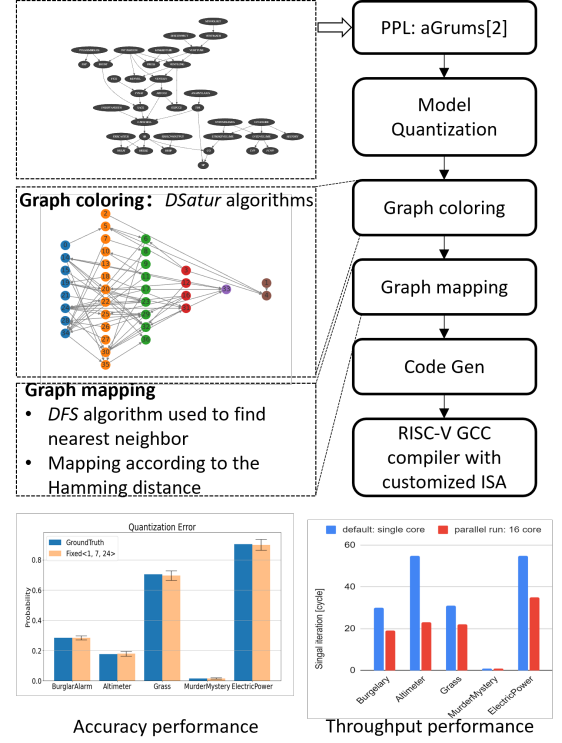


Fig. 5: Customized compiler chain for AIA.

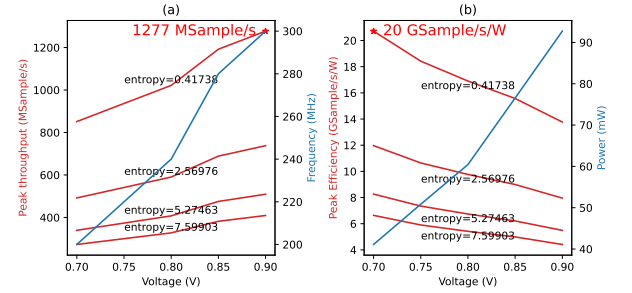


Fig. 6: Peak performance scaling with frequency and voltage. ★ denotes the peak throughput and energy efficiency performance.

coloring stage, with a heuristic that maximizes the parallelism and minimizes the communication distance between nodes that have information exchange. Finally, using the custom RISC-V GCC compiler, a binary is generated for each core to execute the respective probability generation and node sampling.

### IV. MEASUREMENT RESULTS

AIA is implemented in Intel 16nm technology. Fig. 8 shows the die photo with area of  $4mm^2$  and test environment where the memory occupied 66.8% in the area breakdown.

Fig. 6 shows the chip's Schmo plot which is run on sampling from the distributions with different entropies. The peak throughput performance is attained at 300MHz with a voltage of 0.9V, while the highest energy efficiency is reached at a voltage of 0.7V.

Next, Two types of end-to-end applications are mapped to the chip for SotA benchmarking: (a) Bayes Nets [14]; (b)

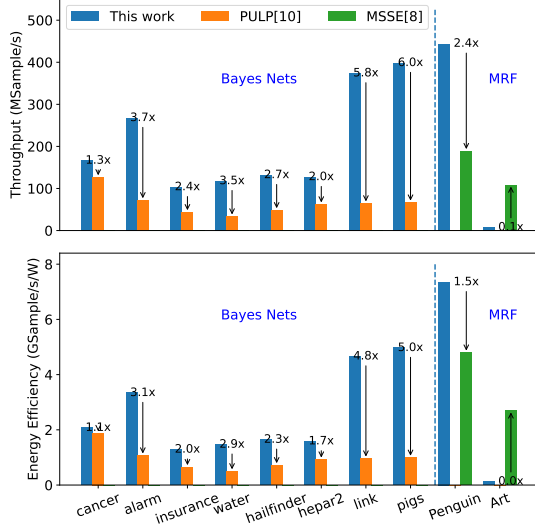


Fig. 7: Throughput and energy performance comparison on real-world applications.

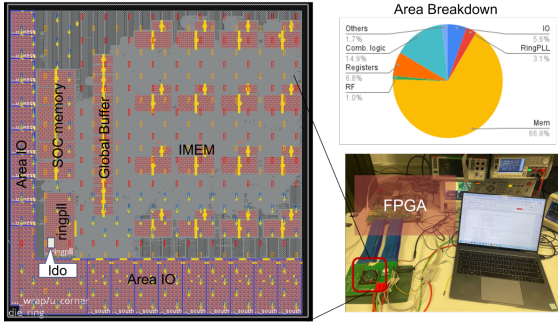


Fig. 8: Chip post layout photo, area breakdown and test environment.

two MRF datasets from [8]: image segmentation for Penguin (image size of 500x333, label size of 2); stereo match for Art (image size of 384x288, label size of 16). As no other SotA ASIC chip supporting Bayes Net approximate inference is currently available, we benchmark our performance against running the same kernels on the multi-core RISC-V PULP platform. Benefiting from our highly efficient sampler and parallel compiler algorithm, it shows superior performance across various Bayes Nets workloads and one MRF workload. Notably, we achieve an average 3.0x increase in throughput and an 2.8x improvement in energy efficiency on the different Bayesian Networks as shown in Fig. 7. However, in scenarios where the cost of distribution generation is dominated, as seen in the "Art" workload, MSSE [8] exhibits better performance.

Table II compares AIA to the SotA, as the only implementation developed based on KY sampling algorithm that can accelerate any MCMC algorithm relying on discrete sampling, our work achieves up to 3.4x higher throughput and 2x higher area efficiency.

TABLE II: Comparison table with SotA

|                                              | This work                                 | MSSE [8]         | SPU [9]    |
|----------------------------------------------|-------------------------------------------|------------------|------------|
| Technology                                   | 16nm                                      | 16nm             | FPGA       |
| Die Area                                     | 4mm <sup>2</sup>                          | 5mm <sup>2</sup> | -          |
| SRAM                                         | 960KB                                     | 103KB            | 4MB        |
| SU No#                                       | 16                                        | 12               | 32         |
| $F_{max}$                                    | 300MHz                                    | 651MHz           | 146MHz     |
| Power@ $F_{max}$                             | 93.5mW@0.9V                               | 42.2mW@0.8V      | -          |
| Peak Throughput (GSample/s)                  | 1.27                                      | 0.372*           | 4.67       |
| Peak Energy Effi. (GSample/s/W)              | 20                                        | 17.6*            | -          |
| Peak Area Effi. (GSample/s/mm <sup>2</sup> ) | 0.567                                     | 0.284*           | -          |
| Sampler                                      | Knuth-Yao                                 | CDF              | CDF        |
| Model                                        | <b>General PMs</b><br>(MRF, BayesNet)     | Only MRF         | Only MRF   |
| Inference                                    | <b>discrete MCMC</b><br>(Gibbs, MH, etc.) | Only Gibbs       | Only Gibbs |

\*: contains both sampling and computing

## CONCLUSION

This work presents a 4mm<sup>2</sup> multicore SoC in 16nm, which executes approximate inference acceleration of PMs. It contains 4x4 customized RISC-V cores which feature non-normalized KY samplers and interpolation unit, as well as core-to-core direct data access via the register file. A customized compiler chain is developed to map MCMC algorithms for a wide variety of PMs. It shows the flexibility and performance gains compared to the SotA implementations.

## ACKNOWLEDGMENT

This project has been partly funded by the European Research Council (ERC) under grant agreement No. 101088865, Huawei, the Flanders AI Research Program and KU Leuven. Special thanks to ETH for the PULP platform support.

## REFERENCES

- [1] H. Wang *et al.*, "A survey on bayesian deep learning," *CSUR*, 2020.
- [2] G. Ducamp *et al.*, "agrum/pyagrum: a toolbox to build models and algorithms for probabilistic graphical models in python." PMLR, 2020.
- [3] N. Gooskar *et al.*, "3dp3: 3d scene perception via probabilistic programming," *NIPS*, 2021.
- [4] S. Hosseini *et al.*, "Bayesian networks for supply chain risk, resilience and ripple effect analysis," *Expert systems with applications*, 2020.
- [5] P. Arora *et al.*, "Bayesian networks for risk prediction using real-world data: a tool for precision medicine," *Value in Health*, 2019.
- [6] N. Shah *et al.*, "Dpu: Dag processing unit for irregular graphs with precision-scalable posit arithmetic in 28 nm," *JSSC*, 2021.
- [7] J. Laurel *et al.*, "Statheros: Compiler for efficient low-precision probabilistic programming," in *DAC*, 2021.
- [8] T. Tambe *et al.*, "A 16-nm soc for noise-robust speech and nlp edge ai inference," *JSSC*, 2022.
- [9] R. Bashizade *et al.*, "Accelerating markov random field inference with uncertainty quantification," *arXiv:2108.00570*, 2021.
- [10] A. Pullini *et al.*, "Mr.wolf: An energy-precision scalable parallel ultra low power soc for iot edge processing," *JSSC*, 2019.
- [11] S. Zhao *et al.*, "Discrete samplers for approximate inference in probabilistic machine learning," in *DATE*, 2022.
- [12] F. Saad *et al.*, "The fast loaded dice roller: A near-optimal exact sampler for discrete probability distributions." PMLR, 2020.
- [13] A. Hagberg *et al.*, "Exploring network structure, dynamics, and function using networkx," Tech. Rep., 2008.
- [14] G. Elidan, "Bayesian Network Repository," 2001.